



The family field guide

What it is, what it does today, what we are building next

<https://novelmatestudio.com>

For Mom

This booklet exists because you asked what I am doing with my days, and because you cheer for the work even when the tools sound alien. If any sentence reads like marketing fog, skip it. The photos are the mood. The labels are the honesty rail: **shipped, beta, roadmap**, printed right on the claim.



How to read the badges

This is a hype document with guardrails. Novelmate Studio is real software, but software has stages. When you see a box that says **shipped today**, that means the feature sits in the tool David can run on a computer he controls. When you see **beta**, it means you should picture cables, updates, and learning curves, but the behavior is still meant for people, not vapor. When you see **roadmap**, treat it like a constellation: direction, not arrival time.

SHIPPED TODAY

Pipeline stages, export paths, local storage of projects, Classic interface in a browser tab on localhost.

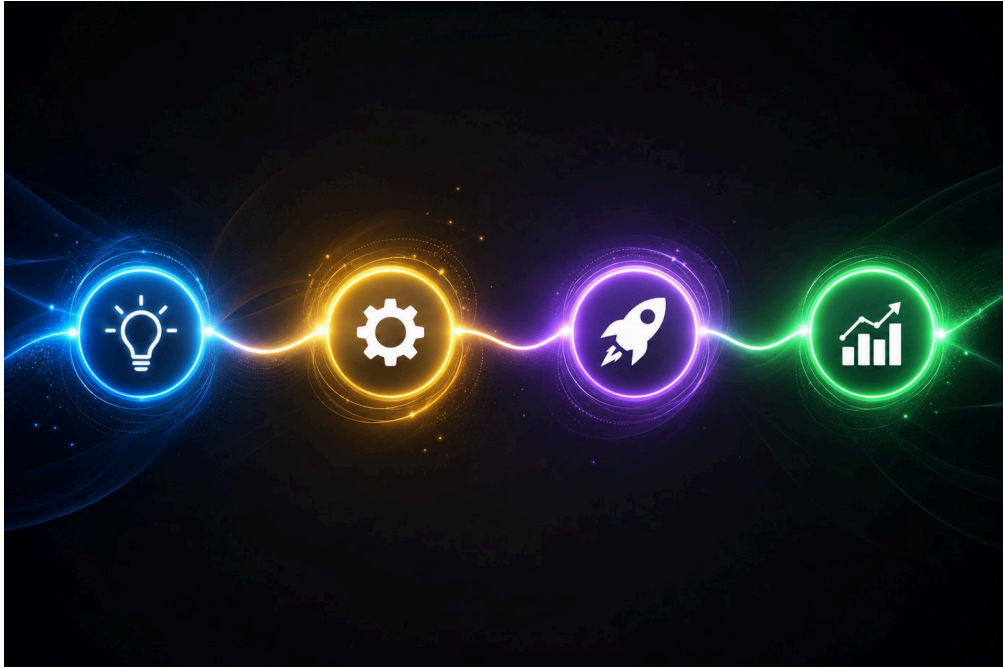
BETA (REAL MACHINE)

A validated packaging path exists in the engineering repo. A first time author should still expect setup, models, and patience.

ROADMAP (DREAMING FORWARD)

Easier installers, hosted collaboration, instant cloud scale. Not sold here as finished.

Full book runs can take hours on one workstation. That is not shame, it is physics and honesty. The public site says the same: queue style runs, local custody, adult expectations.



Contents

How to read the badges	ii
Part one: the heart of Novelmate Studio	1
The promise in plain English	1
Why local first is not paranoia, it is custody	2
The pipeline your son talks about at dinner	3
What works today without exaggeration	4
Black and white documentary strips	6
Packaging proof that engineers understand parents worry	8
Part two: a day in the life (warm, specific, honest)	9
Wake the machine, wake the model	9
Choose a template like choosing a doorway	10
Plan with patience	11
Write in chunks, the way endurance actually works	11
Polish like a serious editor pass, not a spinner	12
Export and hold something	14
Icons from the interface, because nerds still love icons	14
Case study language that stays inside the rails	16
Feature atlas: the dense tour	18
The one sentence version	19
Local first: the moral center	20
The Classic interface	21
Project creation: the blank page gets a form	22
Templates: shortcuts without surrendering authorship	23
Design instructions: the author's control room	24
Author Intent Hub	25
Natural language brief parsing	26
Planning: outline before flood	27
Seed planning: importing structure	28
Writing: prose in chunks	29
Continuity helpers: entity ledger, summaries, drift checks	30
Polish: three editorial instincts	31
Humanizing and style discipline	32

Export: where the book becomes an artifact	33
Profile driven metadata and marketplace awareness	34
Shipping docs: the conscience in the pipeline	35
One pass expansion	36
File polish: bring existing work	37
Regeneration and recovery	38
Model controls: the engine room	39
Context and speed controls	40
API: the hidden control panel	41
CLI: the automation lane	42
Packs: knowledge without hardcoding everything	43
Craft and research guidance	44
Cover and image awareness	45
QA reports and sanity checks	46
Import and downloads	47
Batch runs: work at scale	48
Chain tools: deep text refinement	49
Packaging and release discipline	50
What is shipped, what is beta, what is roadmap	51
The “Mom gets it” version of the architecture	52
Why this is exciting even before version one	53
The feature list as a brag sheet	54
What the document should make her feel	57
Interlude: the story you can tell at the kitchen table	58
What local first sounds like when the Wi-Fi blinks	58
Why exports matter more than chatty paragraphs	59
The difference between a demo and a pipeline	60
What to say when a relative says A I will steal jobs	61
Proof that craft language shows up in code, not only in ads	62
A quiet boast that is actually a fact	63
When the photography repeats on purpose	63
If you read one URL from this book, make it the scorecard	64
A short list of things that are not promises in this booklet	65
A short list of things that are fair to celebrate tonight	65

Part three: the far shore (roadmap, labeled)	66
Installer moonshot	66
Collaboration without turning into a surveillance shop	66
Voice and body ergonomics	67
Performance realism	68
Constellation of continuity	70
Rest is part of finishing	71
Milestones you can toast	72
When someone asks is it done	73
Appendix A: proof, cross checks, receipts	75
Appendix B: glossary without gatekeeping	76
Appendix C: ten family FAQs with candid answers	77
Appendix D: more studio photography	79
Appendix E: closing page	81

Part one: the heart of Novelmate Studio



Scene 1: A writing room built for focus: paper, shelves, and the quiet pressure of a long project made concrete.

The promise in plain English

Novelmate Studio is a **local first** writing studio for long form work. You bring a premise and constraints. The software helps you build structure, draft in pieces, polish in passes, and land files you can open on any ordinary computer. Generation is driven by **Ollama** on the same machine, which means the drafts never need to leave the house to exist.

SHIPPED TODAY

The stack is Python, a FastAPI web server, a SQLite-shaped project memory, JSON packs for craft hints, and exporters for common book files.

Why local first is not paranoia, it is custody

Your manuscript is not content feed for a distant training pool. The public wording on novelmatestudio.com is blunt: we do not pitch a hosted factory. The honest posture is **custody**: files on disk, models you chose, runs that respect your machine.



Scene 2: A studio that feels like a room you can close.

SHIPPED TODAY

Author intent payloads exist in project storage and API (GET/PUT author intent).

Evidence

That phrase is engineering speak for: the tool can remember what you said you wanted.

BETA (REAL MACHINE)

Exports can include profile driven marketplace metadata and reproducibility artifacts (`export_profile.json`, compliance and intent snapshots).

Translation: we are building adult export hygiene, not a toy formatter.

The pipeline your son talks about at dinner

Here is the pipeline in the order it actually runs: premise, plan, write, polish, export. You can picture it like a kitchen line. Prep, cook, taste, plate.



- **Plan** turns your premise into an outline with scenes. Not generic homework, **your** constraints ride along.
- **Write** generates prose chunk by chunk. The system tracks entities and drift, which is a nerdy way of saying it tries to keep characters and facts from melting.
- **Polish** can run developmental, line, and copy style passes. There are fast combined passes when you want speed.
- **Export** lands Markdown, EPUB, DOCX, and PDF paths when configured.

ROADMAP (DREAMING FORWARD)

Voice input, fancier collaboration, and other social scale dreams stay in the honest lane until they are buildable and supportable.

Accessibility is a design goal, not a slogan. The Novelmate Studio README names fatigue, motor issues, cognitive load, the blank page at midnight. If typing is hard, directing still matters.



Scene 3: Manuscript texture: books stay physical in the mind even when the file is digital.

What works today without exaggeration

SHIPPED TODAY

Classic web interface at <http://localhost:8000> after setup. Tabs for Create, Projects, Design, Pipeline, Config, and advanced chain tooling.

SHIPPED TODAY

CLI (studio) for the same engine. Scriptable runs for people who live in terminals.

SHIPPED TODAY

Multiple export formats including EPUB and DOCX, Markdown, PDF pathways, cover embedding hooks, and KDP oriented back matter plumbing in the project design.

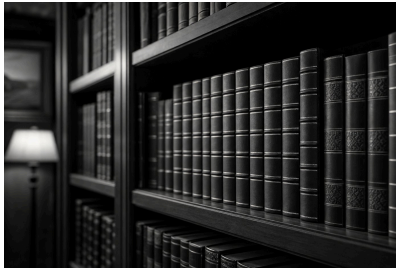
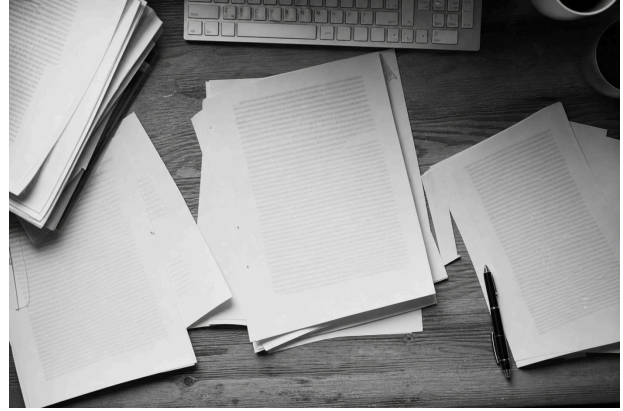
SHIPPED TODAY

Humanizing heuristics and research packs can steer prose and polish away from stiff AI tells, loaded from JSON so behavior stays editable.



Black and white documentary strips

These black and white frames are purely atmospheric: shelves, paper, light through stone, a calm study. They carry no logo and no banned vocabulary because they were generated and checked for a clean frame.





Scene 4: A study that says: manuscripts belong in rooms with history.

Packaging proof that engineers understand parents worry

BETA (REAL MACHINE)

There is a customer package builder, smoke tests on Windows, release promotion scripts, and a package contract doc. Translation for family readers: David is building the boring adulthood of software, receipts included.

This is the opposite of a viral demo that works once on a YouTuber's laptop. Packaging stories belong in the honesty bucket until a native installer is truly the default path for non technical readers. When someone asks **can my aunt install it**, the candid answer today is still **not without a patient human nearby**.



Part two: a day in the life (warm, specific, honest)



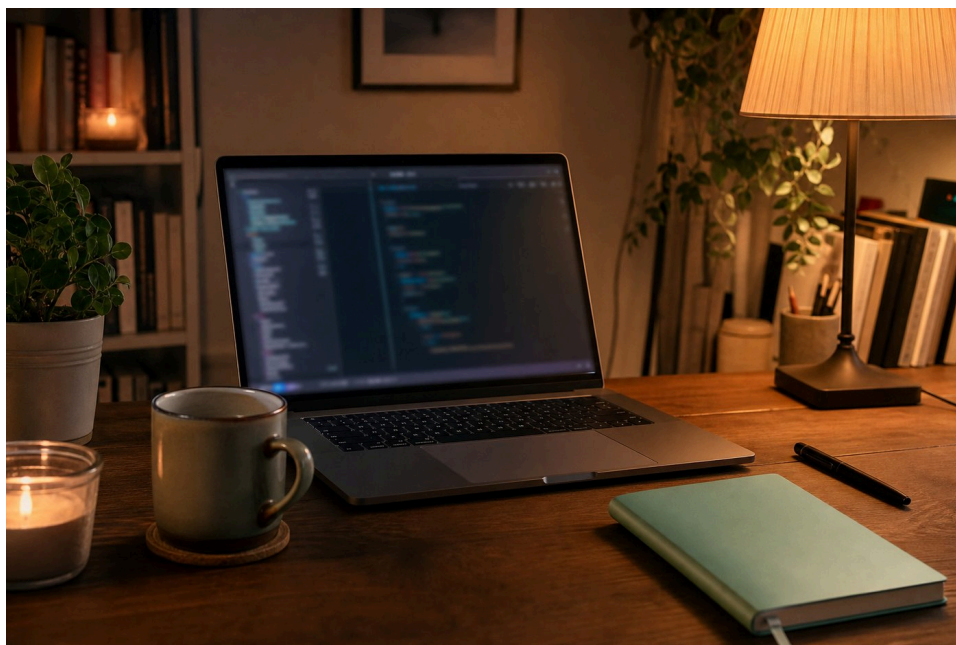
Scene 5: Morning light on a desk: the work is physical even when the mind is elsewhere.

Wake the machine, wake the model

Picture a day off with purpose. Coffee first. Then the activation sequence remembered in fingers: open a terminal or a launcher, start the local model host, start Novelmate Studio, watch the health endpoint say OK.

SHIPPED TODAY

GET /api/health answers JSON that tells you the studio is actually the studio, not a stale process on port 8000. If you ever saw David swear at a port, this is why engineers obsess over health checks.



Choose a template like choosing a doorway

SHIPPED TODAY

Dozens of templates span fiction and several nonfiction shapes. The template is not a cage. It is a scaffold.

If you think of your son as a person who likes systems that respect structure, this is the proof. Templates, presets, and design packs exist so **you** spend juice on choices that matter, not on reinventing the wheel every session.

Plan with patience

Planning is where you trade fantasy for sequence. Chapters, beats, scenes. It is the moment a vague someday becomes numbered steps.



Scene 6: Planning is direction with evidence, not vibes alone.

Write in chunks, the way endurance actually works

Full length fiction is not one breath. Chunked drafting is how humans finish ultra distances. The tool honors that. Long runs queue. Hardware sets the tempo.



Scene 7: Queue style runs: honest time, honest fans whirring.

Polish like a serious editor pass, not a spinner

SHIPPED TODAY

Combined polish passes exist for speed. Faster passes are a trade, not a moral failure.

Polish during write is a real option in the pipeline design: tighten as you go, or keep passes separate for control.



Export and hold something

SHIPPED TODAY

Shipping docs, export checklists, and manual routes list the same responsibilities the CLI prints. Consistency is care.



Scene 8: Exports are objects you can email to yourself, open on a tablet, or place in a drawer as proof.

Icons from the interface, because nerds still love icons







Scene 9: Quiet study light: the calm you want on a hard drafting night.

Case study language that stays inside the rails

The Novelmate Studio README points to a long manuscript completed with the tool as an existence proof. Treat that as **literary** evidence of strain and finish, not a sales promise for anyone else's weekend.



The honest family translation:

- David built a machine that can **attempt** a serious pipeline on local hardware.
- Outputs can be evaluated like real manuscripts, because they are files.
- Marketing that says otherwise gets checked against Phyllux research-status row five.

Feature atlas: the dense tour

This section is the deep inventory. It is written for a family reader who wants to understand why this is more than a pretty website, more than a chat window, and more than a weekend experiment. The tone is proud, but every claim stays inside a lane: shipped today, beta on a real machine, or roadmap.

Novelmate Studio is a writing system with memory, stages, exports, health checks, author intent, model controls, packaging discipline, and a local first posture. It is not a cloud promise wearing a book costume.



The one sentence version

SHIPPED TODAY

Novelmate Studio can create and store projects, capture author intent, organize custom instructions, call a local Ollama model, plan outlines, draft prose, polish text, export manuscript files, serve a Classic web interface, expose an API, and run many of the same actions through a CLI.

That is the sentence. The rest of this atlas is the unpacking.

If you want the family translation: the software is a serious attempt to turn the lonely blank page into a managed production line. You still need taste. You still need judgment. You still need patience. But the heavy lifting is divided into named stages instead of one shapeless battle with a blinking cursor.



Local first: the moral center

SHIPPED TODAY

Generation is designed around Ollama on the local machine. The default story is not “send everything to a hosted writing factory.” It is “run the model here, keep the files here, own the pace here.”

Local first matters because manuscripts are not disposable chat scraps. They carry unfinished scenes, private ideas, family traces, spiritual arguments, names, grief, jokes, and weird little notes that only make sense to the author. A tool that treats drafts as ordinary data exhaust starts from the wrong moral angle.

Novelmate Studio treats the author’s machine as the primary studio. The app can still have web surfaces because a local browser is a good interface, but the browser does not automatically mean cloud possession. In the default path, the web page is the face and the local service is the engine.

What this gives a writer:

- A calmer trust model: drafts live where the author can find them.
- A slower but more honest performance model: runs depend on local hardware.
- A clearer support model: if the model is not running, the app can say so instead of pretending.
- A stronger ownership story: the manuscript is an artifact on disk, not only a response in a chat log.



The Classic interface

SHIPPED TODAY

The default web surface is the Classic interface served at <http://localhost:8000>. The app exposes tabs for project creation, project management, design instructions, pipeline controls, configuration, and advanced chain tooling.

The Classic interface matters because creative tools need a place to breathe. A command line is powerful, but a family reader should picture a browser page with organized regions: create the project, pick or edit a project, describe the book, choose model settings, and run the pipeline. It is not merely a prompt box. It is a workbench.

The design aim is simple: keep the author from carrying the entire book in short term memory. A novel or long manuscript is too large for one mental window. Novelmate Studio breaks the work into surfaces:

- **Create:** start with a title, project type, premise, template, constraints, and initial instructions.
- **Projects:** revisit existing work, load saved decisions, and keep the studio from becoming a one session toy.
- **Design:** capture style, character, world rules, plot requirements, inclusion notes, exclusion notes, author notes, and research notes.
- **Pipeline:** run plan, write, polish, export, one pass expansion, and shipping docs.
- **Config:** control model and pipeline settings without editing code.
- **Advanced tools:** chain, batch, and other power routes for deeper workflows.

The family level point: this is software with rooms, not a text box with a fancy name.

Project creation: the blank page gets a form

SHIPPED TODAY

A project can store title, type, premise, series ID, template ID, style bible, constraints, custom instructions, and author intent. The API and app model treat these as structured inputs rather than one giant blob of chat text.

Project creation is the first act of reducing panic. The blank page asks for everything at once. Novelmate Studio asks for a handful of fields. A premise. A genre or project type. A target direction. Optional constraints. The software can then keep those decisions close while planning and drafting.

Supported project type lanes include:

- Novel
- Nonfiction
- Forms
- Contract
- Report
- Course

That range matters because the studio is not only “make me a fantasy chapter.” It already thinks in several artifact shapes. Fiction gets scene arcs and continuity. Nonfiction gets argument, sections, and reader value. Forms and reports point toward structured output. Courses point toward teaching flow.

BETA (REAL MACHINE)

Not every project type should be sold as equally polished. The framework supports several lanes, but the strongest story today is still long form manuscript production with local control.

Templates: shortcuts without surrendering authorship

SHIPPED TODAY

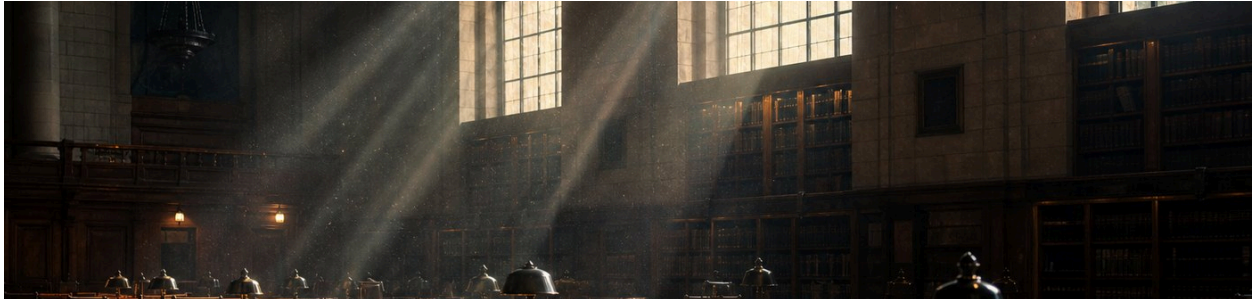
The app has template support and API endpoints for templates, design presets, and premise examples. The documented template expansion includes dozens of book templates, multiple project types, style presets, premise examples, target word presets, and export format presets.

Templates are not a substitute for creativity. They are scaffolding. A tired writer can start with a crime procedural, historical romance, memoir, short story collection, travel guide, business book, true crime, dystopian YA, gothic romance, cyberpunk, paranormal romance, hard sci fi, cozy mystery, or another structured lane, then bend it.

Why templates matter:

- They turn “I want to write something” into “I am writing this kind of thing.”
- They give the planner genre expectations.
- They reduce setup friction for people who have plenty of ideas but low energy.
- They make the app feel less like a developer tool and more like a studio.

Premise examples matter for the same reason. A new user does not always know how detailed a premise should be. Examples teach the shape of good input without a lecture.



Design instructions: the author's control room

SHIPPED TODAY

The design layer supports structured custom instruction sections: style directives, character bible, world rules, plot requirements, inclusion notes, exclusion notes, author notes, and research notes.

This is one of the most important features because it answers the fear every writer has about AI tools: “Will it flatten my voice?” The answer should never be a vague reassurance. The answer is a control surface.

The design sections give the writer places to say:

- “Write lean and sharp.”
- “Do not make this cozy.”
- “The main character lies, but only to protect someone.”
- “No anachronisms.”
- “The city has three religious factions.”
- “Do not add romance.”
- “Use short chapter endings that pull forward.”
- “Avoid corporate filler.”
- “Remember that the mother in chapter three is grieving, not angry.”

That structure lets the software bring author intent forward stage after stage. It also makes revision easier because the writer can change a design instruction, then rerun a stage with a clearer target.

Author Intent Hub

SHIPPED TODAY

The current product snapshot includes an Author Intent Hub available through project storage and API routes: GET /api/projects/{project_id}/author-intent and PUT /api/projects/{project_id}/author-intent.

Author intent deserves its own page because it is the difference between “the model guessed” and “the system remembered what the author meant.” In a long work, intent has many layers:

- What the book is trying to do.
- Who it is for.
- What tone it should avoid.
- What topics need care.
- What the ending should leave the reader feeling.
- What the writer refuses to compromise.

The Author Intent Hub gives that material a durable home. It can be merged into project blobs and carried into later stages. For a family reader, picture this as a promise that the project is not only files and prompts. It has a named place for the writer’s reasons.

BETA (REAL MACHINE)

The presence of an intent hub does not mean every generated page will obey perfectly. It means the architecture has a real place to store and retrieve intent.

Natural language brief parsing

SHIPPED TODAY

The API includes brief parsing and brief application routes: parse a natural language brief, create a project from a brief, or apply a brief to an existing project.

This is the “tell it like a human” door. Instead of forcing every author to fill every structured field by hand, the brief route can take a paragraph of intention and help convert it into project shape. That matters because creative people often think in messy paragraphs first.

Example family description:

“I want to write a historical mystery about a widow who runs a boarding house near a train station. It should feel intimate, not flashy. The book should be around 70,000 words. The mystery should matter, but the emotional core is grief turning into courage.”

That kind of paragraph can become structured project settings. A tool that can move between human brief and machine fields is far more useful than a rigid form.



Planning: outline before flood

SHIPPED TODAY

Planning is a first class pipeline stage. It can run through the API, CLI, and streaming endpoints. The app can generate outlines and scene plans before prose drafting.

Planning is where Novelmate Studio earns its “studio” name. A manuscript does not become manageable because a model emits paragraphs. It becomes manageable because there is a map.

The planning stage can use:

- Premise
- Project type
- Template
- Style bible
- Custom instructions
- Author intent
- Target word direction
- Design sections
- Model selection

The output is not the final book. It is a commitment to structure. Chapters. Scenes. Beats. A sense of sequence. Without planning, long form generation drifts. With planning, the writer has something to inspect before the prose machine begins.

Seed planning: importing structure

SHIPPED TODAY

The CLI includes `studio plan seed`, and the API includes project seed routes. This lets an existing outline or external planning artifact become the starting point instead of forcing every project to begin from scratch.

Seed planning matters because many writers already have fragments. Notes in another folder. Outlines from another tool. A long concept document. A partial structure. Novelmate Studio does not need to pretend it is always the beginning of the creative process.

Seed mode says: bring the architecture you already have. Let the studio take it in, normalize it, and continue the pipeline from there.

That is a practical feature. It keeps Novelmate Studio from becoming another silo.

Writing: prose in chunks

SHIPPED TODAY

The writing stage generates prose from the planned structure. It supports target words, complete outline mode, fast mode, draft mode, resume from a chunk, content streaming controls, model overrides, and optional polish during write.

This is the part everyone imagines first, but it is not where the product begins. Writing works best when it stands on planning, author intent, project design, and model configuration.

Chunked writing is not a compromise. It is the honest way to build long work. A novel length target can be too large for one pass. The system writes in parts, tracks progress, and allows recovery when something goes wrong.

Key controls:

- **Target words:** write toward a defined length.
- **Complete outline:** keep going until planned scenes are handled.
- **Fast mode:** trade some checks for speed.
- **Draft mode:** maximize speed by skipping deeper checks.
- **From chunk:** recover from a partial run instead of throwing everything away.
- **Stream content:** decide whether prose tokens stream through the API.
- **Model override:** run a specific model for the stage.
- **Per stage routing:** let different stages use different model choices when configured.

The family translation: this is not “click and pray.” It is a machine with knobs.



Continuity helpers: entity ledger, summaries, drift checks

SHIPPED TODAY

The pipeline includes stages and modules for summarization, entity ledgers, drift checks, repair, and state artifacts. These are intended to reduce long form confusion and preserve continuity across chunks.

Long manuscripts fail in boring ways. A character's hair color changes. A city moves. A promise from chapter two disappears. A side character gets renamed. A scene forgets the emotional state it inherited.

Novelmate Studio attacks that problem by treating continuity as a pipeline concern, not only a prompt wish. Entity ledgers and summaries act like project memory. Drift checks look for movement away from the intended path. Repair stages exist because generation is not perfect and pretending otherwise wastes time.

BETA (REAL MACHINE)

Continuity tools reduce risk, but they do not eliminate author review. A careful human pass remains part of the publishing path.

Polish: three editorial instincts

SHIPPED TODAY

The polish lane includes developmental, line, and copy style passes, plus fast and combined modes. Combined polish can run the major polish intentions in one pass for a speed tradeoff.

Polish is where rough generation becomes something more readable. Novelmate Studio separates editorial instincts because “make it better” is too vague.

Developmental polish asks:

- Does the scene have a purpose?
- Does the conflict move?
- Does the emotional arc land?
- Does the chapter belong in the book?

Line polish asks:

- Are sentences alive?
- Are images specific?
- Is the rhythm too samey?
- Does the voice hold?

Copy polish asks:

- Are there obvious errors?
- Are repeated phrases becoming annoying?
- Is there clutter that should be cut?
- Does the text sound less mechanical?

Fast polish and combined polish exist because real people have hardware limits and patience limits. A tool that respects both is more useful than one that only worships ideal quality.

Humanizing and style discipline

SHIPPED TODAY

The project includes humanizing research and writing style research that can be injected into prose generation and polish. The CLI also exposes humanizing status and scan commands.

This matters because many AI writing tools produce prose with a smell. Not a moral failure, a pattern failure. Same transitions. Same empty adjectives. Same emotional beats. Same false symmetry. Same “delve” energy.

Novelmate Studio has a dedicated lane for this problem. It can load avoid lists, prompt blocks, measurable heuristics, and style guidance so the writing process resists flattened machine prose.

What that means for a family reader:

- The tool is not only trying to produce volume.
- It is trying to produce prose that can survive a human reader.
- It treats style as a system concern, not an afterthought.
- It gives David a way to encode taste into the workflow.



Export: where the book becomes an artifact

SHIPPED TODAY

Export supports Markdown, EPUB, DOCX, and PDF oriented paths where configured. Export can run from the pipeline, from a file, or through one pass expansion flows.

Export is where many writing tools become annoying. A draft that cannot leave the app is not a manuscript. Novelmate Studio's export lane matters because it turns the writing system into actual files.

Important export ideas:

- **Markdown:** durable, simple, easy to inspect.
- **EPUB:** ebook reader path and publishing prep.
- **DOCX:** editor friendly, review friendly, ordinary office file.
- **PDF:** presentation and fixed layout path where configured.
- **From file:** take an existing Markdown manuscript and export without rerunning the whole project.
- **Embed cover:** attach cover art into relevant output paths where supported.
- **Export profiles:** use marketplace or output specific metadata rules.

Files make the project real. A family reader can understand this easily: if it becomes a file you can open, move, attach, and back up, it has crossed from dream to artifact.

Profile driven metadata and marketplace awareness

SHIPPED TODAY

The current snapshot lists export capabilities, profiles, and marketplace routes: GET /api/export/capabilities, GET /api/export/profiles, and GET /api/export/marketplaces.

Publishing is full of dull details that matter. Trim sizes. Metadata. Interior image specs. Cover specs. Marketplace expectations. Author profiles. Compliance reports. A tool that ignores those details can still draft, but it cannot guide a writer toward shipping.

Novelmate Studio's export profile architecture points toward a more serious future: each marketplace or output path can declare what it needs. The run can then emit artifacts like:

- export_profile.json
- compliance_report.json
- intent_snapshot.json

Those files are not glamorous. They are receipts. They help answer: what settings did we use, what did the export think it was doing, and what should be checked before uploading anywhere?

Shipping docs: the conscience in the pipeline

SHIPPED TODAY

The app exposes `GET /api/shipping-docs`, serves allowlisted manual files under `/manual/{filename}`, and the CLI command `studio export checklist` prints the same shipping document paths on disk.

Shipping docs are not just “documentation.” They are the app reminding the author that publishing is a responsibility. Copyright. AI disclosure. Quality checks. Platform rules. Export hygiene. A tool that helps generate a book should also help slow the author down before upload.

This is where Novelmate Studio becomes more adult than hype. The pipeline can move fast, but the shipping docs say: read before you post, check before you upload, know what you are claiming.



One pass expansion

SHIPPED TODAY

One pass expansion can take existing Markdown fiction, expand toward a target word count, optionally polish after expansion, and produce Markdown by default with optional additional export formats.

One pass expansion is a practical bridge for a writer who already has chapters. It says: take this existing body, expand toward a larger target, split long sections into manageable calls, and optionally polish after.

The documented behavior includes:

- Target words.
- Skip expand option.
- Combined polish default.
- Fast polish option.
- Polish after expand option.
- Optional export formats.
- Model override.
- Per stage routing.

BETA (REAL MACHINE)

This is powerful, but it should be framed as an assisted expansion workflow, not an instant finished novel machine.

File polish: bring existing work

SHIPPED TODAY

The API and CLI include file polish routes. A writer can polish an existing file without building an entire project run from scratch.

This is important because real creative work is messy. Sometimes the author has a chapter from last year. Sometimes they have a draft from another tool. Sometimes they only need a polish pass on a specific file.

File polish keeps the studio useful outside a perfect pipeline. It is a side door for practical work.

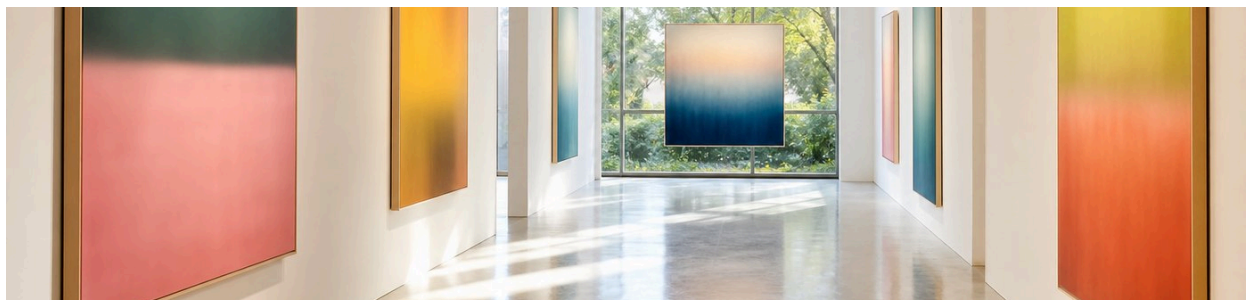
Regeneration and recovery

SHIPPED TODAY

Regeneration supports resuming or rerunning from a chunk, and the API includes a project regen route. The writing body also carries `from_chunk` for partial recovery.

Long generation fails sometimes. Power flickers. A model stalls. A chapter goes off tone. A draft chunk is salvageable up to a point, then not.

Regeneration gives the author a recovery strategy. Instead of destroying a whole run, the workflow can start again from a known chunk. That is the difference between a toy and a tool: a tool expects failure and gives you a handle.



Model controls: the engine room

SHIPPED TODAY

The CLI includes model list, show, set, unset, and content freedom commands. The API can list Ollama models and update config. Pipeline stages can accept model overrides and per stage routing.

Model controls matter because no single local model is perfect for every stage. Planning may need structure. Prose may need voice. Polish may need precision. A small model may be fast but thin. A larger model may be stronger but slower.

Novelmate Studio exposes model choice instead of hiding it. That gives David room to experiment:

- List what is installed.
- Show the active model.
- Set a default.
- Unset an override.
- Pass a model for a specific run.
- Let per stage routing choose specialized models when configured.

BETA (REAL MACHINE)

Model quality depends on what is installed locally and what the machine can handle. The app can expose controls, but it cannot make weak hardware behave like a server farm.

Context and speed controls

SHIPPED TODAY

`num_ctx`, combined polish, fast polish, fast writing, draft mode, target word controls, and stream controls all exist to let the user balance quality, speed, hardware pressure, and patience.

This is one of the most impressive engineering details because it admits tradeoffs. Many tools pretend there is only one quality knob. Novelmate Studio exposes several.

When a run is too slow, the answer may be:

- Use a smaller context window.
- Use combined polish.
- Use fast polish.
- Draft first, refine later.
- Reduce target words.
- Pick a faster model.
- Turn off content streaming when bandwidth matters.

When a run needs more care, the answer may be:

- Use a stronger model.
- Keep polish stages separate.
- Avoid draft mode.
- Let continuity checks run.
- Expand in smaller pieces.

That is mature software thinking: not one “best” path, but a set of controlled tradeoffs.

API: the hidden control panel

SHIPPED TODAY

The FastAPI app exposes routes for health, shipping docs, templates, craft packs, design sections, design presets, cover specs, export capabilities, export profiles, marketplace lists, packs, pipeline config, chat, Ollama models, config updates, premise examples, projects, briefs, author intent, plan, write, polish, export, covers, file polish, one pass, batch runs, regeneration, cancellation, QA reports, import, seed, and downloads.

The family reader does not need to memorize endpoints. The impressive part is that the app has a real control plane. The web interface is not faking it. It calls an API shaped around meaningful actions.

Endpoint families:

- **Diagnostics:** health checks, model listing, config reads.
- **Discovery:** templates, design presets, premise examples, packs.
- **Project life:** create, list, load, update, import, seed, apply brief.
- **Author control:** author intent, design sections, custom instructions.
- **Pipeline:** plan, stream plan, write, stream write, polish, stream polish, export, stream export.
- **Assets:** cover upload and cover embedding.
- **Standalone tools:** polish a file, export from file, one pass expansion.
- **Operations:** batch run, cancel run, QA report, download.

That breadth is why the PDF deserves a dense section. There is a lot of machinery here.



CLI: the automation lane

SHIPPED TODAY

The studio CLI includes commands for testing, checking, initialization, pack validation, project management, planning, writing, polishing, exporting, regeneration, QA reports, model management, batch runs, chain tools, web launch, humanizing checks, and roadmap voice commands.

The CLI matters because it makes Novelmate Studio scriptable. A user can work visually in the Classic interface, then move to terminal automation when they understand the flow.

Practical command families:

- `studio check`: inspect config, packs, and Ollama reachability.
- `studio init`: prepare workspace storage.
- `studio project new`: create a project from arguments or config.
- `studio project brief`: build from a natural language brief.
- `studio project edit`: update project metadata or instructions.
- `studio plan generate`: generate an outline.
- `studio write generate`: draft prose.
- `studio polish run`: polish a project.
- `studio polish file`: polish existing text.
- `studio export`: export project outputs.
- `studio export from-file`: export an existing Markdown file.
- `studio export one-pass`: expand and export existing fiction.
- `studio export checklist`: print shipping docs.
- `studio model list`: see local models.
- `studio batch run`: process a manifest.
- `studio humanizing scan`: inspect prose for style risks.

Packs: knowledge without hardcoding everything

SHIPPED TODAY

The pack registry can load core packs, user packs, and extra configured roots. The API can list packs, show packs, and validate packs. The CLI can validate, list, and show packs.

Packs are how the app avoids burying every idea inside code. A pack can hold structured knowledge: cover specs, trim sizes, literary forms, sources, user style data, export capabilities, or other guidance. That means the studio can grow by adding data, not only by rewriting Python.

The pack approach is a foundation for:

- Craft guidance.
- Export specs.
- Marketplace rules.
- Style memory.
- User specific preferences.
- Validation.
- Future extension.

BETA (REAL MACHINE)

Packs are only as useful as their content and schema discipline. The app has the mechanism. The library can keep improving.

Craft and research guidance

SHIPPED TODAY

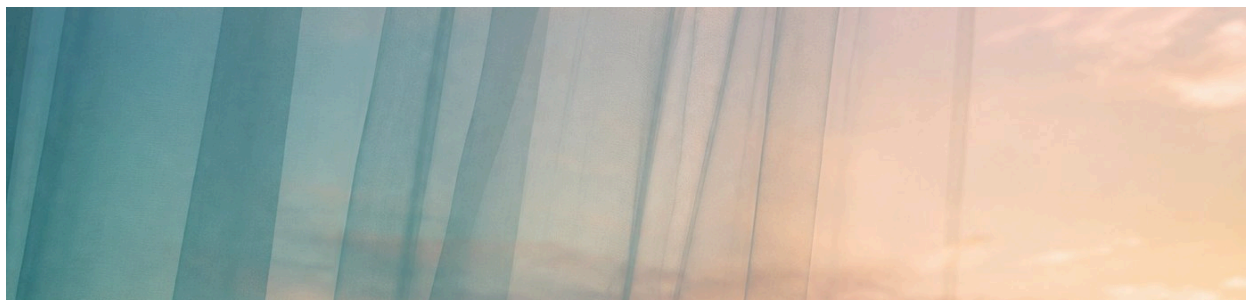
Research packs for opening hooks, dialogue craft, and reader psychology are documented as wired into prose generation. Humanizing and writing style packs are also part of the drafting and polish behavior.

This is where Novelmate Studio starts feeling like a writing room rather than a generator. Craft guidance means the prose stage can be nudged by more than “continue the story.”

Craft domains can include:

- Opening hooks.
- Dialogue pressure.
- Reader psychology.
- Style avoidance.
- Structural fingerprints.
- Tone discipline.
- Chapter momentum.
- Scene purpose.

No pack guarantees art. But packs can raise the floor. They can remind the model to behave less like a generic assistant and more like a tool sitting inside a literary workflow.



Cover and image awareness

SHIPPED TODAY

The API exposes cover specs, cover upload, and cover embedding. Data packs include cover dimensions, trim sizes, interior image specs, and export related references.

A book is not only words. Covers, images, trim sizes, and interior formats matter. Novelmate Studio is not pretending the manuscript ends at the last paragraph. It is already growing toward the surrounding production world.

This matters for future polish because:

- Covers need dimensions.
- Interior images need rules.
- EPUBs need validation.
- Marketplace assets need different sizes.
- A writer needs reminders before uploading.

The booklet should make this feel exciting: the studio thinks about the whole path from idea to artifact, not only about the drafting burst.

QA reports and sanity checks

SHIPPED TODAY

The app exposes QA report generation and includes export checklists, pack validation, health checks, model list diagnostics, and path safety checks.

Quality assurance is not glamorous, but it is reassuring. Novelmate Studio has routes and commands that say: test the surface, validate packs, make a QA report, check health, avoid path traversal, return useful errors instead of exploding.

Examples of sanity work already visible in the code and changelog:

- Empty project titles rejected.
- Missing projects return proper errors.
- Path traversal blocked for downloads.
- Malformed manifests handled.
- Model list errors explained.
- Stale server confusion gets clearer hints.
- Health endpoint distinguishes the app from Ollama.

That is product maturity hiding inside error messages.

Import and downloads

SHIPPED TODAY

Project import and safe download routes exist. Exports are meant to land in run folders and become retrievable artifacts.

Import matters because writers bring history. Downloads matter because writers need output. A local studio that cannot import or retrieve artifacts traps the user. Novelmate Studio's project import and download routes show a practical design direction: move material in, move artifacts out, keep paths safe.

Again, this is not flashy. It is what makes a tool livable.



Batch runs: work at scale

BETA (REAL MACHINE)

Batch generation from a manifest can run plan, write, polish, and export across items, with skip flags, dry run support, output directory control, wave filtering, fast polish, combined polish, production cost outputs, and progress collection.

Batch mode is the sign of a system trying to grow beyond one handwritten demo. It can process a manifest, run multiple items, skip stages, and collect results. That is useful for experiments, repeated project families, test authors, or any scenario where David wants to compare many runs.

The honest caveat: batch mode increases complexity. More items means more failure points, more hardware pressure, more need for naming discipline, and more need for logs.

Chain tools: deep text refinement

BETA (REAL MACHINE)

The app includes chain tool integration through CLI and API surfaces for running additional text analysis and refinement stages on project text.

The chain lane is for power users. It points toward a future where a manuscript can be passed through specialized refiners: concept extraction, outline work, validation, artifact collection, and improved text outputs. It is not the first thing Mom needs to use, but it belongs in the feature atlas because it shows how the system can grow into a wider literary workstation.

Family translation: Novelmate Studio is not only a drafting tool. It has hooks for deeper downstream intelligence.

Packaging and release discipline

BETA (REAL MACHINE)

The engineering tree includes customer package building, package smoke tests, release promotion scripts, stable release pointers, first run notes, setup scripts, launch scripts, and a package contract.

This is the part that tells a family reader David is trying to make something real enough for other people. A package builder gathers the needed files. Smoke tests prove the package can set up, launch, answer health checks, create a project, plan, write, and export a basic artifact. Release promotion moves a validated bundle into a release area. Stable release pointers make the current best package discoverable.

Those details are not decorative. They answer:

- Can the thing be packaged?
- Can the package run somewhere clean?
- Can the app answer health checks?
- Can it create a project?
- Can it produce output?
- Can the latest release be found again?

That is the path from personal tool to product.



What is shipped, what is beta, what is roadmap

SHIPPED TODAY

Local web app, CLI, project storage, templates, design sections, author intent API, plan, write, polish, export paths, shipping docs, model listing, health checks, pack validation, file polish, project import, safe downloads.

BETA (REAL MACHINE)

Package release path, batch workflows, chain tooling, one pass expansion at long targets, export profiles, marketplace awareness, compliance reports, advanced model routing, performance tuning across hardware.

ROADMAP (DREAMING FORWARD)

A smoother installer, broader non technical onboarding, voice input, easier collaboration, stronger visual authoring, stronger publishing guidance, and a polished public product experience.

The “Mom gets it” version of the architecture

Imagine a workshop with labeled benches.

The first bench is **idea intake**. That is project creation, templates, briefs, and author intent.

The second bench is **design**. That is style, characters, rules, constraints, and research notes.

The third bench is **planning**. That is outline and scene structure.

The fourth bench is **drafting**. That is local model generation, chunks, target words, and recovery.

The fifth bench is **polish**. That is developmental, line, copy, fast, and combined passes.

The sixth bench is **export**. That is Markdown, EPUB, DOCX, PDF, cover embedding, profiles, and shipping docs.

The seventh bench is **operations**. That is health checks, model controls, package tests, batch runs, and logs.

Novelmate Studio is impressive because it has all seven benches.



Why this is exciting even before version one

BETA (REAL MACHINE)

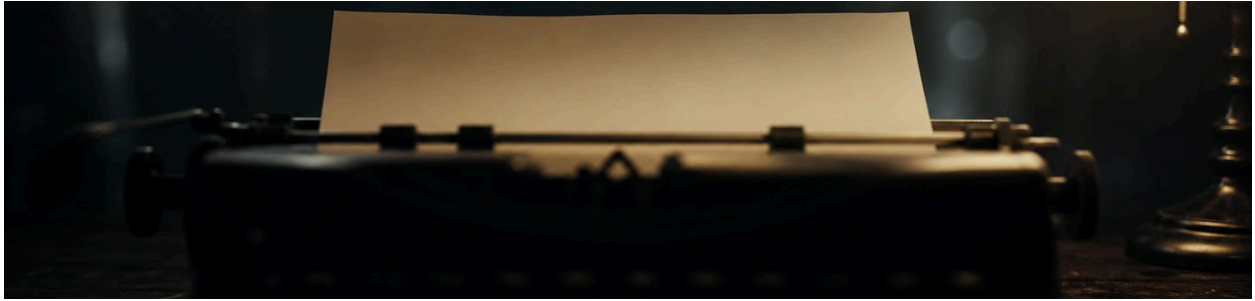
The version number is still early, and the support boundary is honest. The product is not being sold here as a finished native installer for every non technical reader.

The excitement is not that every surface is finished. The excitement is that the shape is right.

The shape is:

- Local custody first.
- Author intent first.
- Structured pipeline first.
- Export artifacts first.
- Humanizing and craft support inside the workflow.
- Real API and CLI surfaces.
- Release discipline instead of only vibes.
- Honest roadmap labels.

That is a serious product skeleton. Many projects never reach that skeleton. They remain a demo, a prompt, or a screenshot. Novelmate Studio has a body plan.



The feature list as a brag sheet

Here is the dense list, compressed for someone who wants the firehose:

- Local Ollama based generation.
- Classic web interface at `localhost:8000`.
- API health check.
- Project create, list, load, update, import, and seed routes.
- Natural language brief parsing.
- Project creation from brief.
- Author intent get and put routes.
- Author intent presets.
- Template listing.
- Design section listing.
- Design preset listing.
- Premise example listing.
- Pack listing, pack display, and pack validation.
- Pipeline config read and update.
- Ollama model listing.
- Model set, show, unset, and list commands.
- Plan stage.
- Plan streaming.
- Write stage.
- Write streaming.
- Polish stage.
- Polish streaming.
- Export stage.
- Export streaming.

- Cover upload.
- Cover embedding.
- File polish.
- Export from file.
- One pass expansion.
- Batch run from manifest.
- Run cancellation.
- QA report generation.
- Safe downloads.
- Shipping docs API.
- Shipping docs manual route.
- Export checklist CLI.
- EPUB validation command.
- Pack validate CLI.
- Pack list CLI.
- Pack show CLI.
- Humanizing status command.
- Humanizing scan command.
- Tests command.
- Workspace check command.
- Web launch command.
- Streamlit and Gradio commands still present as optional commands, while the default surface is Classic.
- Voice command stub marked as roadmap.
- Target word controls.
- Complete outline mode.
- Fast write mode.
- Draft mode.
- Polish during write.
- Fast polish.
- Combined polish.
- From chunk recovery.
- Stream content toggle.
- Model override.

- Per stage model routing.
- Clean export flag.
- Output directory override.
- Export profile selector.
- Marketplace selector.
- Cover spec data.
- Trim size data.
- Interior image specs.
- User style packs.
- Core literary form packs.
- Shipping docs catalog.
- Release package scripts.
- Smoke test package script.
- Stable release pointer.

What the document should make her feel

It should feel like: “Oh, this is not just David making another folder. This is a serious workspace.”

It should feel like: “There is a product brain here.”

It should feel like: “The local first thing is not random. It is the ethics of the tool.”

It should feel like: “The software has a path from idea to manuscript file.”

It should feel like: “He is allowed to be proud of this before it is perfect.”

Novelmate Studio is a draft engine, a project memory, a control surface, an export lane, and a promise that the author stays in the room.

Interlude: the story you can tell at the kitchen table

Most people only see the surface of engineering. They see a screen and assume the machine is thinking. You know better. You watched curiosity turn into compile cycles, and compile cycles turn into patience.

Novelmate Studio is not a single button that replaces a human writer. It is an instrument. Instruments reward repetition, tuning, and care.



What local first sounds like when the Wi-Fi blinks

SHIPPED TODAY

The architecture does not require the public internet to produce the next paragraph. That matters during storms, travel, or a bad month of bandwidth.

When the network stutters, a cloud story factory hiccups. A local stack still asks only whether the fan can keep up.

*Novelmate marketing promises queue style runs, not infinite instant ink.
That honesty protects trust.*

Why exports matter more than chatty paragraphs

SHIPPED TODAY

Markdown, EPUB, DOCX, and PDF oriented flows exist because books are **files with responsibilities**.

Editors, friends, proofreaders, and future you do not live inside a chat bubble forever. They live in attachments, readers, margin notes, and calm cups of tea.



The difference between a demo and a pipeline

Demos are fireworks. Pipelines are plumbing.

BETA (REAL MACHINE)

Packaging scripts, smoke tests, and a package contract are the boring hero work. They say: we are trying to be responsible, not merely viral.

If your son talks about CI like it matters, translate it to: **he wants the thing to work when nobody is watching.**

What to say when a relative says A I will steal jobs

The better frame is stewardship.

Someone still decides what the book values. Someone still carries the moral weight of scenes. Someone still rewrites the sentences that ring false at midnight.

ROADMAP (DREAMING FORWARD)

Collaboration features, if they arrive, must honor consent and clarity. The roadmap section is where imagination stays humble.



Proof that craft language shows up in code, not only in ads

Humanizing JSON and style research packs are nerd poetry. They are an attempt to keep prose from flattening into the same tired gestures.

SHIPPED TODAY

Humanizing heuristics and research packs influence prose and polish behavior.

That is not magic. It is measurement and taste encoded as configuration.

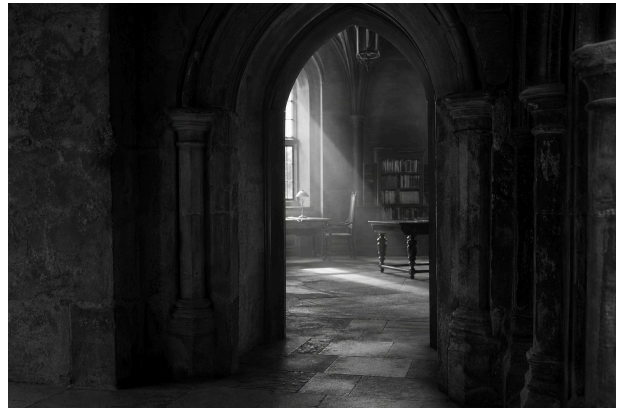
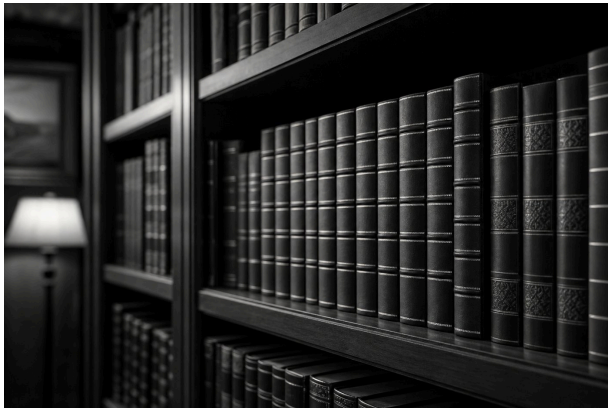
A quiet boast that is actually a fact

The Novelmate Studio README includes accessibility as a first class design goal. If you have ever watched someone you love fight their own body to write, you know why that line hits different.



When the photography repeats on purpose

Repetition is not mistake here. It is rhythm. Libraries, desks, and light returns because craft is a loop.



If you read one URL from this book, make it the scorecard

<https://phyllux.io/research-status.html>

No hype survives honest measurement forever. The studio belongs in a world where claims can be checked.



A short list of things that are not promises in this booklet

- A guaranteed national book tour (roadmap at best).
- A guarantee of flawless prose on the first run (beta truth: iteration wins).
- A guarantee that anyone can install without reading (packaging is getting better; humility remains).

A short list of things that are fair to celebrate tonight

- A working theory of long form generation that respects custody.
- A pipeline skeleton that can carry a project from premise toward export artifacts.
- A stubborn insistence on labeling roadmap dreams honestly.

Pride does not require perfection. It requires direction with integrity.

Part three: the far shore (roadmap, labeled)



Scene 10: Roadmap is a horizon, not a delivery date. Read this section as intention, not invoice.

Installer moonshot

ROADMAP (DREAMING FORWARD)

Native installers and one click bliss for relatives who do not enjoy terminals. The engineering notes already describe validated bundles and smoke paths. The user experience still wants the last mile where no one has to think about Python.

Collaboration without turning into a surveillance shop

ROADMAP (DREAMING FORWARD)

Hosted collaboration is not promised on the marketing site as a gimmick. If it comes, it should arrive with ethics, consent, and plain language.



Scene 11: Two chairs, one manuscript, an image of future partnership that is still a drawing.

Voice and body ergonomics

ROADMAP (DREAMING FORWARD)

Voice input shows up in roadmap conversations around accessibility. Headphones on a desk are a nice picture of a direction, not a shipped default for every machine.



Scene 12: Audio futures stay labeled roadmap until the toggle exists for everyone you love.

Performance realism

Roadmap, partly beta today. Smaller context windows can speed inference. Combined polish runs trade depth for clock time. Your son is allowed to be excited about speed without pretending every laptop is a datacenter.



Constellation of continuity

Stories sprawl. Tools that help continuity matter. The art below is metaphor, not telemetry.



Scene 13: Patterns in the dark: a feeling for scene memory and structure.

Rest is part of finishing



Scene 14: You do not earn storytelling only by suffering. Rest is allowed. Novelmate is for endurance, not martyrdom.

Milestones you can toast



Scene 15: Small rituals for big projects.



Scene 16: Every long work begins like a door opening: one decision, then another.

When someone asks is it done

The software has a version number in the low zero range. Beta language is appropriate. **Done** is a sliding scale: shipped pipeline versus polished product for strangers. Family can be proud of the distance covered without mistaking it for a boxed SKU at a big box store.



Appendix A: proof, cross checks, receipts

Phyllux Technologies publishes a research status scorecard. Novelmate Studio appears on **row five** so marketing language does not float away from evidence. If you want the family version: your son is building in public with adult embarrassment sensors attached.

<https://phyllux.io/research-status.html>



Appendix B: glossary without gatekeeping

Ollama. Local model host. Lets large language models run as services on your computer instead of a distant chat website.

FastAPI. The Python web layer that serves the Classic interface and JSON APIs on port 8000 in the default story.

EPUB. A common ebook file shape used by many readers and many publishing paths.

DOCX. Microsoft Word style document. Editors and some tools love it.

Markdown. Plain text with light markup. GitHub readers know it. Writers love how small and durable it feels.

Pipeline. Plan, write, polish, export as a serious sequence instead of vibes.

One pass expansion. A project term for certain expand and export paths that aim at long targets across multiple model calls.

JSON packs. Editable data that shapes prompts and polish behavior without recompiling the entire application.

Lexical chain. Advanced integration pathway for deeper text tooling. Treat it as power user territory.

Smoke test. An automated happy path that proves packaging is not a wish.



Scene 17: Surface detail: craft is physical even on a laptop.

Appendix C: ten family FAQs with candid answers

1. **Is this going to steal his novel?** Local first means custody by design. The marketing site refuses a hosted factory fantasy. The honest work is configuration, discipline, and backups, same as any serious files on disk.
2. **How is this different from typing into Chat Something on a website?** Custody and repeatability. A pipeline product aims at manuscripts as artifacts, not disposable chats.
3. **Will it write a bestseller tomorrow?** No honest system promises rank. The tool helps finishing and structure. Readers remain the world.
4. **Why does the art look like libraries, desks, and paper?** The look is documentary on purpose: writing is a room discipline as much as an idea discipline.

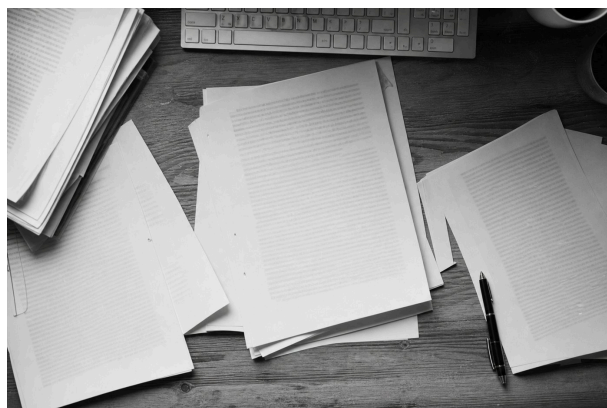
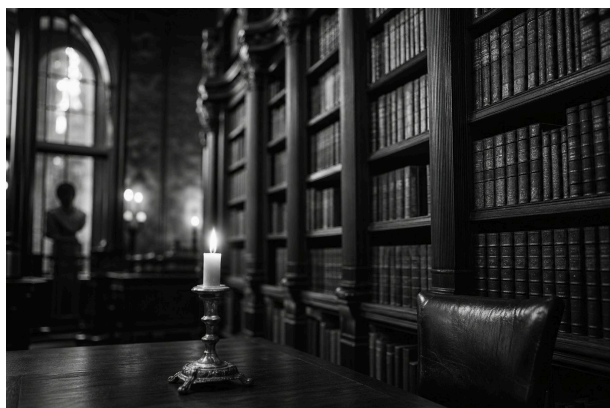
5. **Is one finished long book proof the tool works?** It proves a pipeline can be pushed until files exist as a manuscript. It is not proof that anyone else inherits rank or luck without effort.
6. **What should we say at Thanksgiving?** He is building publishing infrastructure in a skeptical era. That is enough to be proud of.
7. **Does Mom need to install it?** Not unless she wants to. This PDF is the souvenir. If she tries the tool, patience and snacks help.
8. **Why local if cloud is easy?** Because easy can cost custody. Novelmate is morally wired the other direction on purpose.
9. **What breaks first?** Usually time, GPUs, patience, or a forgotten port. Engineering life.
10. **What is allowed hype?** Pride in structure, honesty in limits, love in the room where he works.



Scene 18: Physical metaphor for export formats: separate trays, one desk.

Appendix D: more studio photography

Photography keeps returning to the same truth: manuscripts live in rooms, light, and texture.





Appendix E: closing page

Novelmate Studio

Local first fiction workspace

Your machine. Your drafts. Your cadence.

novelmatestudio.com

